

COLM 2024 | 大模型真的会「组合」技能吗？

威斯康星大学麦迪逊分校研究：LLM 组合能力的二分现象与规模化边界

我们早已习惯大模型的一项本领：只要在上下文里给几个例子，它就能现学一个新任务，这就是所谓的上下文学习（in-context learning）。但真实世界的推理很少只是重复一个操作，更多时候是把几个已经掌握的技能拼在一起用。

于是有一个很自然、却几乎没被系统检验过的问题：如果一个模型分别学会了任务 A 和任务 B，当它遇到一个需要同时用到 A 和 B、而且从没见过的组合任务时，能不能自动把两个技能拼起来？对人类来说这几乎是本能，但对大模型而言，答案远没有那么理所当然。

这篇发表于 COLM 2024 的论文给出的答案，是一个相当清晰的「二分现象」（dichotomy）：当两个子任务作用在输入的不同部分时，模型能够很好地组合它们，而且模型越大表现越好；可一旦任务需要真正的多步链式推理，模型往往直接失败，把模型规模做大也救不回来。

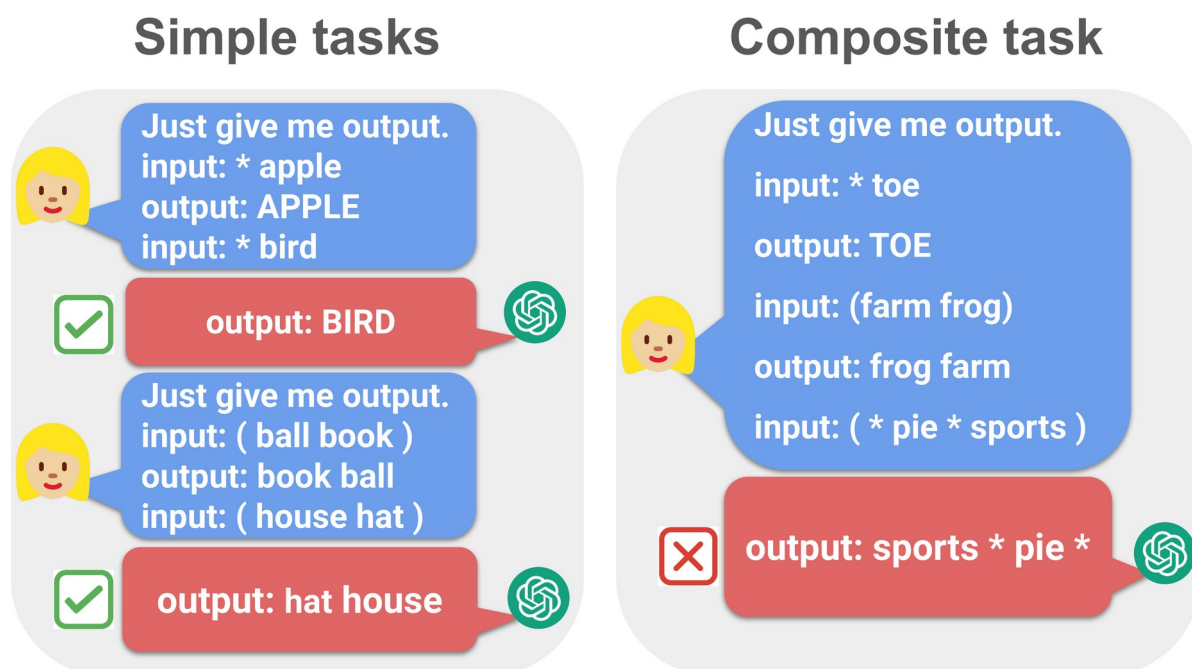


图 1：组合式上下文学习示意。给定简单任务的示范（* 后面的词要大写，括号里的词要交换），模型能分别解出每个简单任务，却在需要同时应用两者的组合任务上出错，给出了错误的输出。

论文地址：<https://arxiv.org/abs/2407.15720>
地址：https://github.com/OliverXUZY/LLM_Compose

代码地

一个人类觉得理所当然、模型却常常做不到的任务

先看一个具体例子。给模型两组简单示范：带星号 * 的词要变成大写 (capitalization)，括号里的两个词要前后交换 (swap)。分别测试时，模型两个任务都做得又快又好。可如果给它一个同时含有星号和括号的输入，要求两个操作一起完成，即便是 GPT-4 和 Claude 3 这样的前沿模型，也常常翻车。

关键在于：每个单项技能模型都会，出问题的只是「把两者组合」这一步。这说明失败并非来自能力不足，而是来自组合这个动作本身。论文正是围绕这个反差，用一套受控的实验去回答：大模型到底有没有真正的组合能力，还是说这种能力其实比它们的单项表现暗示的要局限得多。

如何测量「组合能力」：四种设定

作者在标准的上下文学习框架下做测量：每次给模型 $K=10$ 个示范样例，外加一个待解的测试输入。围绕每个组合任务，他们设计了四种对照设定，用来把「会不会单项」和「会不会组合」彻底拆开。

前两种是两个简单子任务各自单独测；第三种是组合设定 (composite) ——示范全部来自简单子任务，但测试输入是需要两步的组合任务；第四种是组合内示范设定 (composite_compose_incontext) ——示范和测试都直接换成组合任务，代表模型「如果直接教它组合，它能做到多好」的黄金上界。四种设定放在一起，就能判断模型缺的到底是能力，还是组合。

二分现象：可分离任务 vs 逐步组合任务

论文最核心的发现，是组合任务并非铁板一块，而是分成截然不同的两类。第一类是「可分离」(separable) 组合任务：两个子操作作用在输入的不同部分，彼此不干扰，比如把某些词大写、同时对另一部分做加法。第二类是「逐步组合」(compose-by-step) 任务：两个操作必须串联成一条多步推理链，共享同一段输入，比如先大写再交换。前者模型能组合、且随规模变好，后者模型往往崩溃。

下面这张图展示的正是逐步组合的典型例子：大写与交换 (Capitalization & Swap)。横轴是模型规模，纵轴是精确匹配准确率，四条线对应上面提到的四种设定。请留意红色的组合曲线相对其他三条线所处的位置。

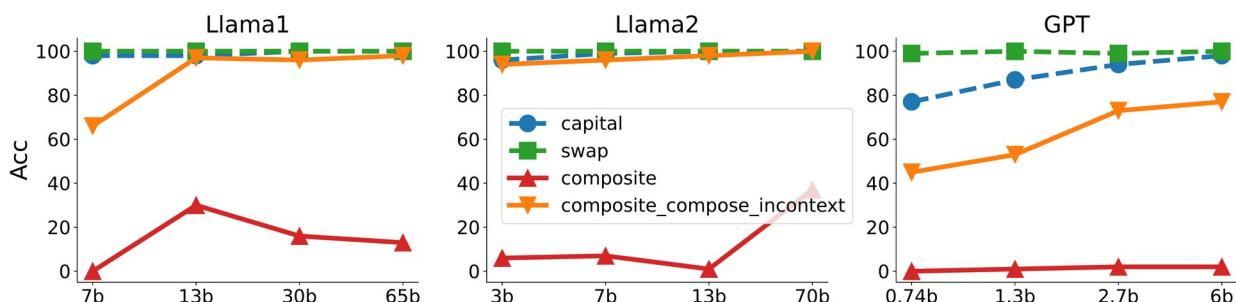


图 2：大写与交换（Capitalization & Swap）任务上，准确率（纵轴）随模型规模（横轴，b 表示十亿参数）的变化。capital、swap 为两个简单任务；composite 为「简单任务示范 + 组合测试」；composite_compose_incontext 为示范与测试都是组合任务的上界设定。

结果非常刺眼：capital 与 swap 两条线几乎贴着 100%，说明单项技能模型完全掌握；组合上界（橙线）也很高，说明只要直接教组合，模型做得到。可真正的组合曲线（红线）却长期趴在 20% 上下甚至更低，而且从 7b 一路放大到 65b、70b，这条线几乎不动——在逐步组合任务上，把模型做大并不能带来改善。

换个任务族看，情况就不一样了。下图是翻译类组合任务，纵轴换成词错误率 WER（越低越好），涵盖短语重组加长链（Phrase Recombination with Longer Chain）与被动转主动加宾主转换（Passive-to-Active and Object-to-Subject）两组。这里的组合更接近可分离的一端。

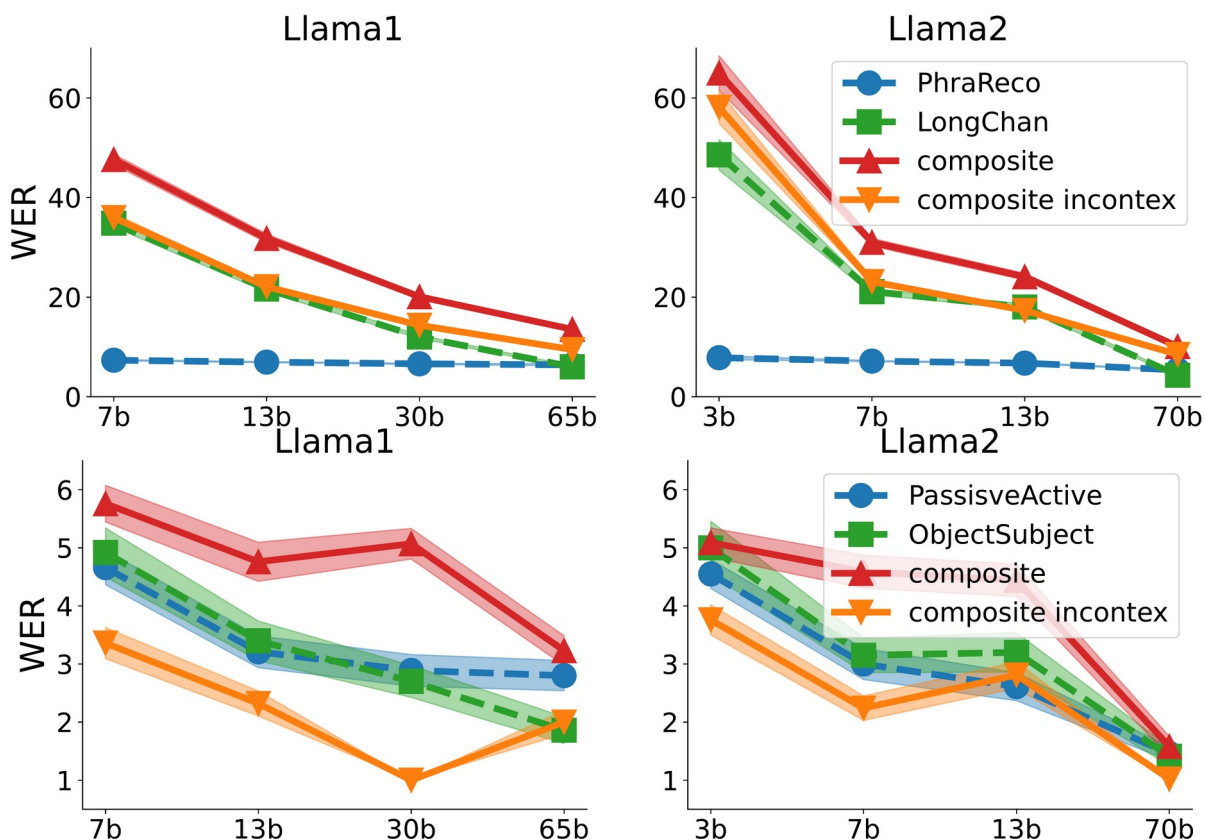


图 3：翻译类组合任务上，词错误率 WER 随模型规模的变化。虚线为简单任务，实线为组合任务；行对应 (T1) 短语重组加长链、(T2) 被动转主动加宾主转换；列对应不同模型；不同线条代表不同评测设定。

在翻译类任务里，组合曲线（红色实线）的 WER 随着模型变大稳步下降，逐渐逼近组合上界（橙线），也贴近简单任务。这正是二分现象的另一半：当子任务在输入上足够分离时，规模化确实能把组合能力带起来。是否值得为一个组合任务堆更多算力，很大程度上可以由任务本身的结构提前判断。

为什么会这样？线性自注意力给出的解释

论文没有停在现象层面，而是给出了理论解释。作者分析了一个单层、单头的线性自注意力（linear self-attention）网络，证明组合能否成功，取决于两个简单任务是否具有「受限支撑」（confined support）——也就是它们在输入嵌入中是否各自占据互相独立的特征子空间。

当两个任务的子空间彼此分离（可分离任务），模型可以在互不干扰的前提下把两者叠加，组合就能成立，而且随着单项任务因规模化而变强，组合表现也跟着变好；一旦子空间发生重叠（逐步组合任务），两个操作在同一段表示里互相纠缠，组合就会崩溃，堆参数也无济于事。理论因此精确地对上了实验里观察到的二分与规模化规律。

关键数字一览

表 1：不同设定下的代表性结果（Llama / GPT 系列，K=10 上下文示范）。

设定	结果
简单任务（大写 / 交换），Llama	约 90%
逐步组合（大写 & 交换）	≤ 20%，随规模基本持平
大写 + Two Sum（较大规模）	44%
交换 + Two Sum（较大规模）	66%
GPT-4 / Claude 3 的组合任务	同样失败

评测覆盖 Llama-1（7b–65b）、Llama-2（3b–70b）、Llama-3 与 GPT（0.74b–6b）等多个规模。

结论与边界

这篇论文的结论既清晰又实用：大模型只有在两个技能作用于输入的不同部分时，才能可靠地把它们组合起来；一旦任务真正需要链式的多步推理，模型会在组合这一步失败，而单纯增大参数量并不能把它救回来。

值得强调的是边界所在。给模型直接提供组合示范时，准确率立刻回升，说明模型并不缺乏表示能力，缺的是「从简单示范里自行拼出组合」的能力。因此，一个组合推理任务能否靠更多算力受益，很大程度上由它的结构决定：可分离（受限支撑）的任务能随规模成长，深度纠缠的多步任务则不能。这为「哪些组合任务可以信任大模型去做」提供了一条可操作的判断线索。